LLM ENGINEERING

Bonus chapter 3

# Bonus: GPU Kernels & the CuTe DSL

Self-paced reading

Interactive version: https://llm.strueber.dev/#/bonus-03-kernels

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

# Why Kernels

1

## Same softmax, same GPU:

### why are five PyTorch ops
### 4.9× slower than one kernel?

Answer A1

2

## Orientation

| When | Submission | Reflection questions |
|---|---|---|
| After Week 5, before Week 6 | None ( `scripts/kernels/` optional) | Three, discussed in the Week 6 tutorial |

- Trace grid → cooperative thread array (CTA) → warp → thread
- Size residency and bytes in flight
- Map a thread-value layout to row offsets
- Explain the 4.9× from passes and bandwidth
- Trace fused softmax, connect it to FlashAttention

▼ Two ideas to bring from Weeks 4 and 5

This chapter is required reading between Week 5 and Week 6, with no submission. It ends with three reflection questions, which the Week 6 tutorial discusses. Prepare a short answer to each. The question slides along the way are self-checks: think about the question first, then reveal the answer.

Two earlier ideas carry the chapter. From Week 4, the roofline: a kernel's time obeys `T ≥ max(F/P_peak, B/β)`, so an operation with few FLOPs per byte of HBM traffic is limited by the bytes it moves. From Week 5, mixed precision: tensors are stored in bf16, with 8 exponent bits and 7 mantissa bits, while sensitive reductions such as sums are accumulated in fp32.

The folder `scripts/kernels/` holds the kernel this chapter walks through, a correctness check and a benchmark. Running them is optional and needs an NVIDIA GPU with the CuTe DSL installed. We ran them on UC3's H100 for the measurements given here. Reading and explaining a kernel is the goal. Writing a production kernel from a blank file is a different skill.

## A kernel runs on thousands of threads

- **Kernel**: one function, run by every thread
- Launch: one call, a grid of threads
- PyTorch GPU operation: one or more kernels
- `torch.softmax`: one kernel per call

▼ **Kernels, launches and where a custom kernel can help**

A GPU kernel is a function that runs on the GPU. The host program on the CPU launches it over a grid of threads, and every thread runs the same function, using its own index to choose its data. One launch starts thousands of threads.

An operation on CUDA tensors in PyTorch queues one or more kernel launches. An operation that only changes metadata, such as a view, launches none. On UC3's H100, for a softmax over a 32,768 × 4,096 bf16 tensor, the PyTorch profiler counted one kernel per call of `torch.softmax`, and five for the same softmax written as five separate operations.

Standard calls such as `torch.softmax` and matrix multiplication already run optimized kernels, so rewriting one of them in a lower-level language rarely helps by itself. The opportunity is usually a composition boundary: several operations launch separately, and each writes an intermediate tensor to HBM that the next reads back, although the data could have stayed on the chip. Week 4 showed that FlashAttention wins by never writing the attention score and probability matrices to HBM (Dao et al., 2022). The row softmax is a simpler operation with the same kind of boundary.

## Five operations move four times the bytes of one

$$\text{traffic ratio} = \frac{8 \text{ passes (decomposed)}}{2 \text{ passes (fused)}} = 4\times \tag{1}$$

- Eager chain: 5 reads, 3 writes
- Fused: 1 read, 1 write
- Measured 4.9×: unequal bandwidths

| UC3's H100, 32,768 × 4,096 bf16 | Time | Kernels |
|---|---|---|
| Eager chain | 1,248.0 µs | 5 |
| `torch.softmax` | 466.8 µs | 1 |
| Fastest fused kernel (course `512x8`) | 253.5 µs | 1 |

▼ Counting passes, and when the ratio can exceed 4

A pass is one full read or one full write of the tensor in HBM. Write a row softmax as five operations: the maximum `m = amax(x)`, the shift `x - m`, the exponential, the sum `s`, and the division by `s`. Run eagerly, each is its own kernel. `amax` reads `x`. The subtraction reads `x` and writes `x - m`. The exponential reads that and writes `exp(x - m)`. The sum reads the exponentials, and the division reads them again and writes the output. That is five reads and three writes, eight passes. (The maximum and the sum also write one value per row, which is negligible.) A fused kernel reads `x` once and writes the output once, two passes. Formula (1) divides the two counts.

When does fusion save a factor of 4 in time? Assume that every intermediate goes to HBM and back, which holds here because one tensor is 268 MB and the L2 cache is far smaller, that every pass runs at the same bandwidth, and that nothing else costs time. Then each version's time is its pass count times the time of one pass, and the ratio is 8 / 2 = 4. Relaxing the assumptions moves the ratio in known directions. Intermediates that stay in L2 make the eager chain faster, and arithmetic time, which both versions pay, dilutes the ratio, so both lower it. Extra kernel launches and unequal bandwidth raise it.

The table gives medians from UC3's H100 for a 32,768 × 4,096 bf16 input (slide 27 has the full table and the protocol). The eager chain is 1,248.0 / 253.5 ≈ 4.9 times slower than the fastest fused kernel. Launches do not explain the difference: the device time of one profiled eager call, 1,257.4 µs, is close to its median, so little time passes between its kernels. Bandwidth does. If the eager chain moves the eight passes counted above, 8 × 268 MB ≈ 2.15 GB in 1,248.0 µs, it averages about 1,721 GB/s, 81% of the 2,132.9 GB/s that a plain copy reaches on this card. The

fused kernel moves its two passes at 2,118.0 GB/s, 99.3% of the copy. The byte ratio times the bandwidth ratio gives the measured gap: 4 × 2,118.0 / 1,721 ≈ 4.9.

`torch.softmax` is itself one fused kernel with one read and one write, but it reaches only 53.9% of the copy's bandwidth, so the eager chain is only 1,248.0 / 466.8 ≈ 2.7 times slower than it. Moving fewer bytes and moving them at full speed are separate properties. The traffic count tells you what fusion can remove, and a benchmark what a kernel achieves.

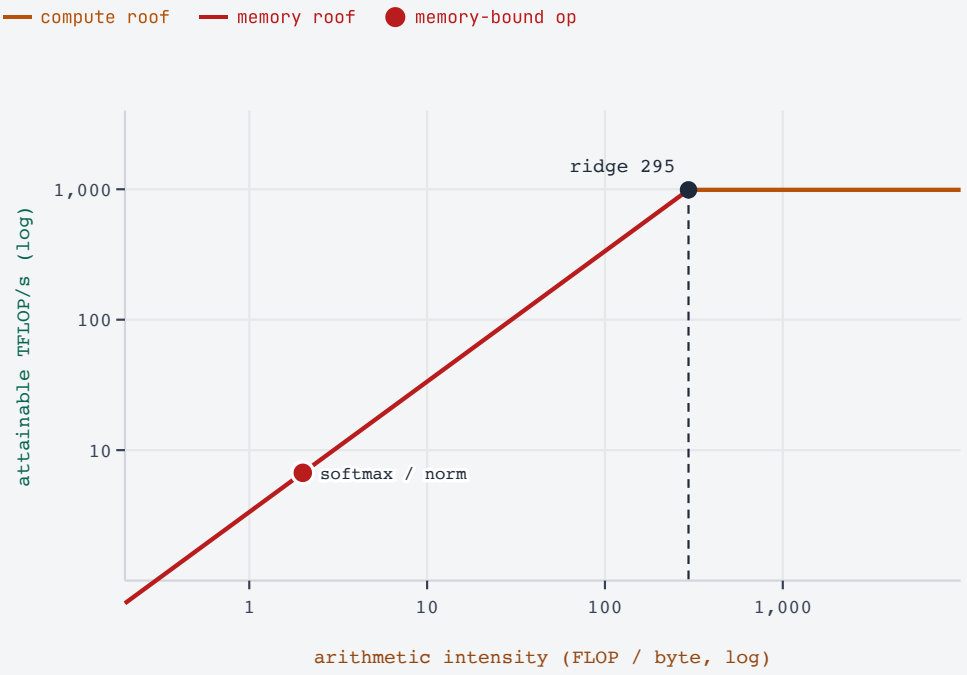
# Softmax falls far below every ridge

- About 1.25 FLOP/byte: 5 operations, 4 bytes
- Drawn roof: bf16 Tensor Core peak, ridge 295
- fp32 roof: about 15× lower, ridge 20
- Chart marker: teaching value 2

**H100 SXM, softmax**

Where does a streamed reduction fall?

peak ≈ 989.5 TFLOP/s (dense bf16) · bandwidth ≈ 3.35 TB/s · ridge ≈ 295 FLOP/byte




The drawn roof is the bf16 Tensor Core peak, with its ridge at 295 FLOP/byte. Softmax cannot use Tensor Cores and runs mostly on the fp32 units, whose roof is about 15 times lower, with a ridge near 20 FLOP/byte (67 TFLOP/s ÷ 3.35 TB/s). The marker at 2 FLOP/byte, a teaching value, lies left of both ridges, so removing full-tensor passes is the lever.

Exact intensity depends on shape, dtype, cache traffic and fusion.

Illustrative

## ▼ Softmax's intensity, its roof, and when fusion does not pay

Per element, a read-once, write-once softmax performs about five operations (a comparison for the maximum, a subtraction, an exponential, an addition for the sum, a multiplication by the reciprocal) and moves 4 bytes, 2 read and 2 written in bf16. That is about 1.25 FLOP per byte. It is an estimate, because the exponential runs on the special-function unit (SFU) and FLOP conventions differ. The chart marks softmax at 2 FLOP/byte, a teaching value it shares with Week 4.

The roof drawn in the chart is Week 4's: the H100 SXM's dense bf16 Tensor Core peak of 989.5 TFLOP/s, with its ridge at about 295 FLOP/byte. Softmax has no matrix product, so it cannot use Tensor Cores. Its own roof is the fp32 peak of 67 TFLOP/s (NVIDIA, 2022), about 15 times lower, with a ridge at 67 / 3.35 = 20 FLOP/byte. The marker lies left of both ridges. For the tensor of slide 4, our own arithmetic gives at least $5 \times 134 \times 10^6 / (67 \times 10^{12}) \approx 10$ µs for the arithmetic and at least 160 µs for moving 268 MB in and 268 MB out at 3.35 TB/s. Removing full-tensor passes is the lever.

Fusion does not always pay. A small tensor can stay in the L2 cache (50 MB on the H100 SXM), so its intermediates never reach HBM, the traffic count overstates the saving, and launch overhead dominates. A compute-bound operation, such as a large matrix multiplication, is limited by its arithmetic, and removing bytes changes little. A chain of launches in a profile is only a candidate. The case is strong when the chain is memory-bound, its intermediates reach HBM, and the compiler leaves the boundary in place.

## Library and compiler before a custom kernel

- Path: library → `torch.compile` → profiler → custom kernel
- Compiler: fuses pointwise and reduction chains
- Picks GEMM backends, one in the CuTe DSL (Patel et al., 2026)
- Custom kernel: only for a profiled boundary
- Done: correctness check, then fair benchmark

▼ The escalation path and what ends it

Start with the library implementation where one exists: it is already tuned and covers many shapes and dtypes. Profile the eager program to find a candidate, a chain of launches whose intermediates travel through HBM. Then try `torch.compile`. Its default backend, TorchInductor, fuses chains of pointwise and reduction operations. For matrix multiplications it can choose among specialized GEMM backends, one of them written in the CuTe DSL (Patel et al., 2026). On UC3's H100, `torch.compile` turned the five-kernel eager softmax into one kernel (slide 27). Profile the compiled program again and see what remains. Write a custom kernel only for a boundary that is still there, and only when the time it saves justifies maintaining it. FlashAttention-4 is a kernel designed together with its algorithm for one hardware imbalance (Zadouri et al., 2026).

A custom kernel starts from a hypothesis, which boundary it removes and how many bytes that saves, and ends with two pieces of evidence. The first is a correctness check against a trusted reference, run before any timing. The second is a benchmark that gives the kernel and its comparators the same shapes, dtypes, outputs, synchronization and measurement protocol. A faster time on a different shape, dtype or looser output is no win.

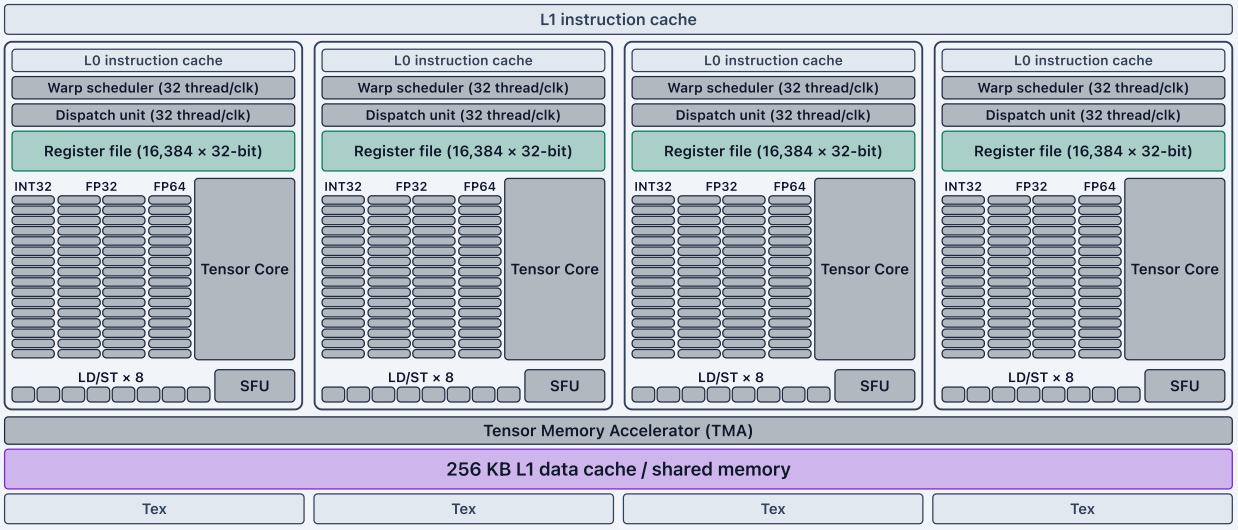
# Execution Model

7

## Each SM runs warps on four sub-partitions

GPU SM

One SM of 132 · 4 sub-partitions




**Registers**
256 KB per SM
bandwidth not given · a few ns

**Shared memory**
up to 228 KB per SM
31 TB/s peak, whole GPU · about 10–20 ns

**L2 cache**
50 MB
12 TB/s peak · about 150–200 ns

**HBM3**
80 GB
3.35 TB/s · about 400 ns

**Execution units**
warp schedulers, cores, Tensor Cores, SFUs, LD/ST units and the TMA

Hover over a part, or tab to it, to see what it does.

After Andersch et al., 2022, Figures 3 and 4

One streaming multiprocessor (SM) of an H100, redrawn after NVIDIA's figure with its four sub-partitions in one row instead of a 2 × 2 grid. Each sub-partition has an instruction cache, a warp scheduler and a dispatch unit, a register file, INT32, FP32 and FP64 cores, a Tensor Core, load/store (LD/ST) units and a special-function unit (SFU). The register files and the L1 data cache and shared memory take their memory tier's colour from the legend, and the units a kernel's warps execute on are marked in grey. Below the sub-partitions the SM has one Tensor Memory Accelerator (TMA), the L1 data cache and shared memory, and the texture units. The drawing does not keep the SM's proportions. The GPU button shows where the SM sits on the chip.

- Four sub-partitions, one warp scheduler each
- A warp stays on one sub-partition
- FP32 cores, special-function unit (SFU), Tensor Cores

### ▼ Inside one SM

Week 4's figure showed an H100 as 132 streaming multiprocessors (SMs) around a shared L2 cache and off-chip HBM. The same figure opens here on one SM, and its GPU button returns to the whole chip.

A few terms first. A thread runs the kernel's code. A warp is a group of 32 threads that execute together, and each thread is one lane of its warp. A cooperative thread array (CTA) is a group of up to 1,024 threads that run on one SM and share memory. Slides 9 to 12 return to each of these levels.

Each SM is partitioned into four sub-partitions. Each has a warp scheduler, a register file and its own execution units: integer and floating-point cores, load/store units, a special-function unit (SFU) and a Tensor Core. The four share the SM's L1 data cache and shared memory and its texture units (NVIDIA, 2026b). On the H100 a sub-partition has 32 FP32 cores, 16 INT32 cores, 16 FP64 cores and one Tensor Core, a quarter of the SM's 128, 64, 64 and 4 (Andersch et al., 2022). Its register file holds 16,384 32-bit registers, 64 KB (Andersch et al., 2022).

A warp is allocated to one sub-partition and stays there from launch to completion (NVIDIA, 2026b). Its threads' registers live in that sub-partition's register file. An SM holds at most 64 warps at a time (slide 9), so, by our own count, each scheduler has at most 16 to choose from. On every cycle, each scheduler picks one warp that is ready to issue and issues its next instruction or instructions (NVIDIA, 2026b). A warp that waits for data from memory is not ready, so in the meantime the scheduler issues from another warp. The slide "Bytes in flight hide memory latency" builds on this.

The softmax kernel uses only some of these units. Its loads and stores go through the load/store units, the maximum, the subtraction, the sum and the scaling run on the floating-point cores, and the exponential runs on the special-function unit (slide 25). It has no matrix product, so it never uses a Tensor Core, and the bf16 Tensor Core roof of slide 5 is not its roof. The Tensor Memory Accelerator (TMA) and the warpgroups (groups of four warps) that FlashAttention uses come in the To FlashAttention tab.

## Data lives in four memory tiers

Global memory (HBM) Registers Shared memory

| Tier | H100 SXM size | Shared by | Rough latency |
|---|---|---|---|
| Registers | 256 KB per SM (64K × 32 bit) | one thread | lowest |
| Shared memory | up to 228 KB per SM | one CTA | about 10–20 ns |
| L2 cache | 50 MB | all SMs | about 150–200 ns |
| HBM | 80 GB at 3.35 TB/s | all SMs | about 400 ns |

- Week 4's SRAM: registers and shared memory
- Latencies: rounded, from the QuACK note (Guo et al., 2025)

▼ **Four memory tiers, and the card you run on**

Week 4's figure, and the SM view on slide 7, place these tiers on the chip. The table adds what a kernel writer needs: who can see each tier. It gives the H100 SXM, the course's reference GPU from Week 4, with 132 SMs (Andersch et al., 2022). Each SM has 64K 32-bit registers, 256 KB, and a register belongs to one thread, at most 255 per thread (NVIDIA, 2023), and other lanes of its warp can read it only through a shuffle (slide 11). Shared memory is shared by the threads of one CTA. Each SM has 256 KB of combined L1 cache and shared memory, of which up to 228 KB can be shared memory, and one CTA can address up to 227 KB. The L2 cache holds 50 MB and serves all SMs (NVIDIA, 2023). HBM, the off-chip memory, holds 80 GB and delivers up to 3.35 TB/s (NVIDIA, 2022). Week 4's on-chip "SRAM" is, on an H100, a kernel's registers and shared memory.

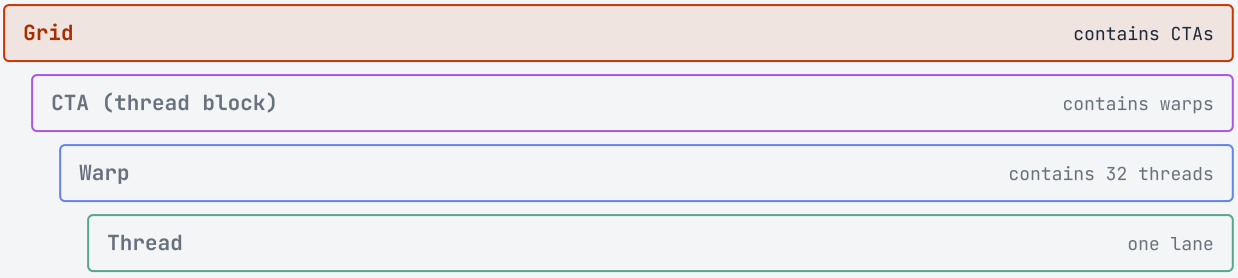
The latencies are the rounded figures the QuACK note gives for an H100 80 GB (Guo et al., 2025). Each tier down is larger and slower to reach, so a kernel that keeps its working data in registers and shared memory touches HBM only for its input and output. The legend's colors mark global memory (HBM), registers and shared memory in the kernel's code and diagrams, as a second cue next to CuTe's names (slide 23).

Always name the exact card. The H100 PCIe has 114 SMs instead of 132 (Andersch et al., 2022), and UC3's card differs from both. `nvidia-smi` reports it as "NVIDIA H100" with 95,830 MiB of memory and a maximum memory clock of 1,593 MHz. The CUDA driver reports a 6,016-bit memory bus, and PyTorch's device properties report 132 SMs and a 60 MiB L2 cache. NVIDIA publishes no bandwidth for this card, so we computed its theoretical peak: 2 transfers per clock × 1,593 MHz × 6,016 bit / 8 = 2,395.9 GB/s, about 2.4 TB/s against the 80 GB SXM's 3.35 TB/s. A plain copy of a 268 MB bf16 tensor reached 2,132.9 GB/s, 89.0% of that peak.

## Threads nest in warps, CTAs and a grid

- At most 1,024 threads per CTA
- Per SM: 64 warps, 2,048 threads, 32 CTAs

CONTAINMENT: LAUNCH → LANE




**Grid**

EXECUTION SCOPE
one kernel launch

SCHEDULED ON
CTAs distributed across available SMs

STATE
kernel arguments and global memory (HBM) tensors

COOPERATION
separate launches or explicit grid synchronization

Launching many tiny grids adds launch overhead.

H100 SXM anchors (NVIDIA Hopper architecture blog, Nsight Compute guide, CUDA Programming Guide)

**132** SMs | **4** warp schedulers / SM | **64** max active warps / SM | **32** threads / warp

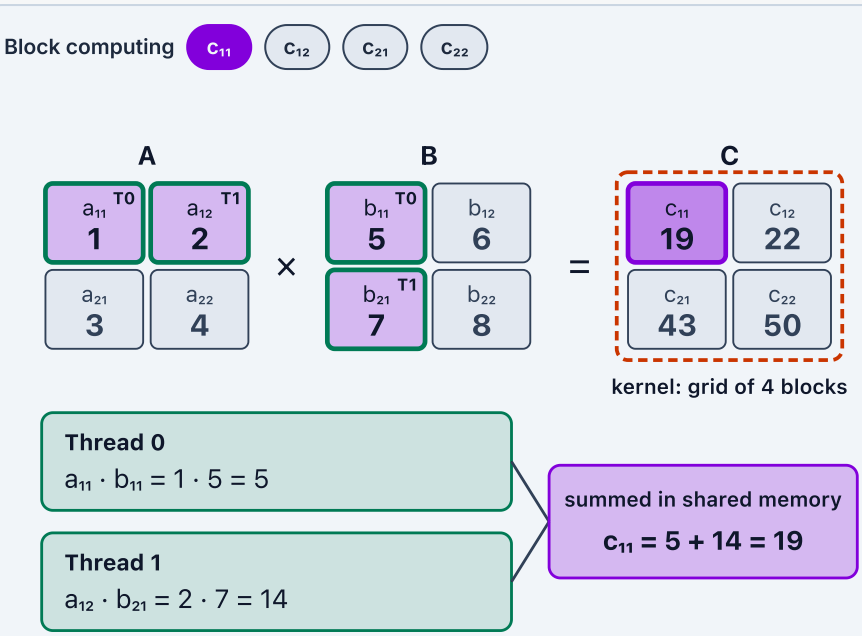
per SM: up to 32 CTAs, 64 warps, 2,048 threads

### ▼ Reading the execution hierarchy

A launch creates a grid of CTAs, each CTA holds warps, and each warp holds 32 threads. Select a level in the widget to compare its execution scope, the hardware that schedules it, its state and how its members cooperate.

The footer gives the H100 SXM anchors, among them the four warp schedulers of slide 7, one per sub-partition. For compute capability 9.0, the H100's, a CTA has at most 1,024 threads, and an SM holds at most 32 CTAs, 64 warps and 2,048 threads at a time (NVIDIA, 2026a). Hopper adds two optional levels. A cluster groups several CTAs scheduled together on a group of SMs (Andersch et al., 2022), and a warpgroup groups four warps (NVIDIA, 2026c). The softmax kernel uses neither.

# Kernel, block and thread split a 2×2 product




| Level | In this example | Runs on |
|---|---|---|
| **Kernel (grid)** | all of C: 4 blocks | the GPU, its blocks spread over the SMs |
| **Block (CTA)** | $c_{11}$: row 1 of A, column 1 of B | one SM, start to finish |
| **Warp** | both threads, 30 of 32 lanes idle | one sub-partition |
| **Thread** | one product | a CUDA core (FP32 lane), one instruction at a time |

A worked example, C = AB for two 2 × 2 matrices with A = [[1, 2], [3, 4]] and B = [[5, 6], [7, 8]]. The dashed frame around C is the kernel's work, a grid of four blocks. The purple cells are the selected block's: one row of A, one column of B and the entry of C they produce. T0 and T1 mark the entries each of its two threads multiplies. Choose a block above, or click an entry of C. The table maps each level to the hardware it runs on.

- Kernel → GPU: all of $C$
- Block → SM: one entry $c_{ij}$
- Thread → CUDA core: one product
- Tensor Cores: one warp multiplies a whole tile

## ▼ A matrix product split into blocks and threads

### The example

Take $C = AB$ for two 2 × 2 matrices, $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Each entry of $C$ is the dot product of a row of $A$ and a column of $B$:

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j}, \qquad C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Split this work over the levels of slide 9. The kernel is the whole computation: one launch produces all four entries of $C$, as a grid of four blocks. Block $(i, j)$ reads row $i$ of $A$ and column $j$ of $B$ and produces $c_{ij}$. It has two threads: thread 0 computes $a_{i1}b_{1j}$ and thread 1 computes $a_{i2}b_{2j}$. For $c_{11}$ these are $1 \cdot 5 = 5$ and $2 \cdot 7 = 14$. Choose a block in the widget, or click an entry of $C$, to see which entries its threads multiply. The table beside the matrices maps each level to the hardware it runs on.

The two products must then be added, and each lives in its own thread's registers. Threads of one block can exchange values through the block's shared memory, with a barrier so that each reads only after the other has written. This is what a block is for: its threads share the block's shared memory and barriers. Threads of different blocks can exchange data only through global memory or, on Hopper, within a cluster (slide 9), which this example does not use. The softmax kernel's CTA combines its 32 warp partials the same way (slide 24). With only two threads, both in one warp, a shuffle (slide 11) would also work.

### From levels to hardware

The kernel runs on the whole GPU. The hardware hands its blocks to SMs as SMs become free (slide 12), so a grid with enough blocks keeps all 132 SMs busy. Ours has four blocks and leaves 128 SMs idle.

A block runs on one SM from start to finish, because its threads share that SM's shared memory. One SM can hold several blocks at once, up to 32 (slide 9), so many blocks can share one SM, but one block never spreads over several SMs.

A thread runs on a CUDA core, NVIDIA's name for an FP32 unit, of which an H100 SM has 128 (Andersch et al., 2022). This holds for one instruction at a time. A warp's 32 threads issue together on one sub-partition (slide 7), and an FP32 instruction gives each thread one of the sub-partition's 32 FP32 cores for that instruction. A thread is not tied to a core. An SM holds up to 2,048 resident threads but has 128 FP32 cores, so its warp schedulers interleave the warps' instructions, and a thread's integer, load and exponential instructions run on other units: the INT32 cores, the load/store units and the SFU.

The warp, the level between block and thread, maps to a sub-partition. Threads are scheduled in whole warps, so our two-thread block still occupies a full warp, with 30 of its 32 lanes idle.

### Where Tensor Cores change the picture

For a matrix product on an H100, "one thread computes one product on a CUDA core" is not how fast kernels work. A Tensor Core multiplies small matrix tiles, and the instruction that drives it belongs to a group of threads. With `mma` (matrix multiply-accumulate), one warp multiplies, for bf16, a 16 × 16 tile by a 16 × 8 tile, 2,048 multiply-adds in one instruction, with the operands spread over the registers of its 32 threads (NVIDIA, 2026c). Hopper's `wgmma` is issued by a warpgroup of four warps and produces output tiles of 64 rows and up to 256 columns (NVIDIA, 2026c). No thread multiplies a single pair of entries. A production matrix multiplication kernel also gives each block a large tile of $C$, not one entry, and loops over the shared dimension in chunks staged through shared memory.

The upper levels stay as in the example: the kernel covers all of $C$, each block owns a part of $C$ on one SM, and the arithmetic happens in the warps of that SM. Only the bottom level changes, from single products on CUDA cores to tile products on Tensor Cores. Softmax has no matrix product, so the course kernel runs on the CUDA cores and the SFU, and the CUDA-core picture fits it. Slide 30 shows how FlashAttention puts its two matrix products on Tensor Cores.

11

## A warp issues one instruction for 32 threads

- **Warp**: 32 threads, one instruction stream
- Single instruction, multiple threads (SIMT)
- Shuffles: register exchange between lanes
- Since Volta: independent lane scheduling
- Explicit synchronization, no assumed lockstep

▼ SIMT, shuffles and independent thread scheduling

A warp is 32 threads that one warp scheduler issues together. Each thread is a lane, numbered 0 to 31, and when the warp issues an instruction every active lane executes it on its own registers. NVIDIA calls this model single instruction, multiple threads (SIMT).

Lanes of one warp can exchange register values through shuffles: each lane reads a register of another lane, chosen by a lane offset, without going through memory. The kernel's warp reduction uses five shuffles (slide 24).

When lanes take different branches, the warp runs both paths with the other lanes masked off. Since Volta (compute capability 7.0), every thread keeps its own execution state, so lanes can diverge and reconverge in groups smaller than the warp. The CUDA Programming Guide therefore calls code that assumes lockstep execution invalid and asks for explicit synchronization with `__syncwarp()` (NVIDIA, 2026a). SIMT remains the programming model. Independent scheduling removes only the guarantee that lanes stay together.

Warp-wide operations such as shuffles need every lane they name to reach them. A common mistake is to put a call that reads the warp index inside `if lane == 0`. Reading the warp index uses a warp-wide shuffle, so lane 0 waits for 31 lanes that never arrive, and the kernel hangs. Read the index before the branch, as slide 24 does. If lanes communicate through memory, or diverge before a warp-wide operation, use the synchronization that operation requires.

## Each CTA stays on one SM

$$32 \frac{\text{registers}}{\text{thread}} \times 1{,}024 \frac{\text{threads}}{\text{CTA}} \times 2 \text{ CTAs} = 65{,}536 = 64\text{K registers per SM} \tag{2}$$

- CTA: threads sharing memory and barriers
- Grid: all CTAs of one launch
- Course kernel: 1,024 threads, 32 registers each
- Result: two CTAs per SM

▼ **Residency: how many CTAs fit on one SM**

A CTA, also called a thread block, runs on one SM from start to finish. Its threads share the CTA's shared memory and can wait for each other at a CTA-wide barrier, and the softmax reduction uses both. A grid is all the CTAs of one launch. The hardware places CTAs on SMs as resources become free, so a grid can be far larger than what fits on the GPU at once.

How many CTAs fit on one SM at once, the residency, is set by the tightest limit. The course kernel's 1,024 threads per CTA allow two CTAs under the limit of 2,048 threads per SM. Registers agree: the compiled kernel uses 32 registers per thread, and Formula (2) shows that two CTAs then fill the SM's 64K registers. At 33 or more registers per thread only one CTA would fit. Shared memory does not limit this kernel, which uses 32 fp32 values, 128 bytes, per CTA. Two CTAs of 32 warps also reach the limit of 64 warps per SM.

The softmax of slide 4 launches one CTA per row, 32,768 in all. With 132 SMs holding two each, 264 run at a time, and 32,768 / 264 ≈ 124.1, so the GPU works through the grid in 125 rounds. Launch overhead matters when a program launches many tiny grids. For one large memory-bound grid, the bytes dominate.

## Bytes in flight hide memory latency

$$\text{bytes in flight} = \text{bandwidth} \times \text{latency} \tag{3}$$

$$3.35\ \text{TB/s} \times 400\ \text{ns} \approx 1.3\ \text{MB} \approx 10\ \text{KB per SM} \tag{4}$$

- **Occupancy**: active warps ÷ maximum warps (NVIDIA, 2024a)
- Lever 1: more resident warps
- Lever 2: more bytes per load
- Occupancy: resident warps, not useful lanes

▼ **Little's law for HBM, and the two levers**

A load from HBM takes about 400 ns to return, and HBM can only deliver bytes already requested. Little's law says how many must be outstanding to keep it busy: the bytes in flight, issued and not yet returned, equal the bandwidth times the latency (Formula 3). Volkov applied it to GPUs as "needed parallelism = latency × throughput" (Volkov, 2010). Formula (4) is our own arithmetic for the H100 SXM: about 1.34 MB for the GPU, 10 KB per SM. For UC3's card, at about 2.4 TB/s and the same 400 ns, it is about 0.96 MB, roughly 7 KB per SM.

A kernel has two levers. The first is more resident warps: while one warp waits for its load, the scheduler issues from another, whose loads add to the bytes in flight. Occupancy measures this lever, as the ratio of active warps on an SM to the maximum it supports (NVIDIA, 2024a). Registers per thread, shared memory per CTA and threads per CTA limit it. The course kernel's two CTAs of 32 warps reach 64 warps, 100% occupancy. The second lever is more bytes per load: wider loads, or several loads issued before the first result is needed.

The course kernel issues one 8-byte load per thread, so when both resident CTAs load at once, the SM has 2 × 1,024 × 8 bytes = 16 KB in flight. That is more than the 7 KB per SM that UC3's card needs, yet the kernel reaches only about two thirds of the copy's bandwidth (slide 27). Our reasoning, not a measurement: bytes are in flight only while a CTA loads. Once its load has returned, a CTA reduces in registers and requests nothing until it stores, so averaged over time the SM keeps fewer bytes in flight than the peak of 16 KB. Slide 27 compares a variant that requests 16 bytes per load at the same occupancy.

Occupancy alone does not mean useful work. A large CTA in which many lanes have nothing to do reports high occupancy and low lane utilization, and a kernel with many registers per thread can run faster at lower occupancy when each warp keeps more bytes in flight (Volkov, 2010). Find the bottleneck first.

## Neighboring lanes should read neighboring bytes

- 32-byte sectors: the unit of transfer (NVIDIA, 2026b)
- Cache line: four sectors, 128 bytes
- **Coalesced**: one warp, few sectors
- H100 loads: at most 16 bytes per thread (NVIDIA, 2026c)

▼ Sectors, lines and coalescing

Global memory moves in aligned 32-byte sectors, and an L1 or L2 cache line holds four sectors, 128 bytes (NVIDIA, 2026b). When a warp loads, the hardware fetches every sector that any of its 32 lanes touches. If 32 lanes read 32 consecutive 4-byte words, the warp needs four sectors and uses every fetched byte (NVIDIA, 2026a). Such an access is coalesced. Scattered addresses can cost one sector per lane, moving 32 bytes to deliver 2 or 4. Coalescing is a separate property from occupancy (NVIDIA, 2024a).

The width of each lane's load is a second property. On an H100 one load instruction moves at most 16 bytes per thread, eight bf16 values. 32-byte vector loads need compute capability 10.0 or higher (NVIDIA, 2026c). A wider load fetches the same bytes with fewer instructions, and each instruction puts more bytes in flight. Slide 17 shows how the kernel's layout sets both properties.

# CuTe DSL

15

## The CuTe DSL compiles Python into a GPU kernel

- CuTe DSL: Python domain-specific language for CuTe (NVIDIA, 2024b)
- `@cute.kernel`: device code each thread runs
- `@cute.jit`: host code, or a helper a kernel calls
- `cute.compile`: compile once, reuse the binary
- Layout algebra: only in just-in-time (JIT) compiled functions

▼ **Host code, device code and one compiled binary**

CUTLASS is NVIDIA's open-source C++ template library for high-performance linear algebra, and CuTe is its algebra of layouts. The CuTe DSL is a domain-specific language: it lets you write CuTe code in Python and compiles it into GPU code (NVIDIA, 2024b).

A function decorated with `@cute.kernel` is device code: every thread of the launch runs it. A function decorated with `@cute.jit` and called from Python is host code: it builds the layouts and launches the kernel over a grid. A `@cute.jit` function can also be called from a kernel, as the reduction helper on slide 24 is, and is then compiled into the kernel's device code (NVIDIA, 2024b). Both are compiled just in time (JIT), when the program first needs them. `cute.compile` traces the host function with example arguments and produces one compiled binary. The course script compiles each variant once per device and reuses the binary. It marks the row count `M` as dynamic, so one binary serves any number of rows, which the correctness check confirms for the default variant with 8 and then 1,000 rows.

Layout algebra works only inside these compiled functions, and a compiled function may not `return` from inside an `if` on a run-time value (NVIDIA, 2024b). A loop over `cutlass.range_constexpr` is unrolled at compile time.

Triton, OpenAI's Python language for GPU kernels, gives each program instance a block of values and lets its compiler map those values to threads, which is concise for many pointwise and reduction kernels. CuTe writes that mapping as a readable layout, and the mapping is what this chapter teaches. FlashAttention-4 (Zadouri et al., 2026), the QuACK kernels (Dao-AILab, 2026) and TorchInductor's CuTe DSL GEMM backend (Patel et al., 2026) are written in the CuTe DSL. For ordinary model code the escalation path of slide 6 still starts with the library and `torch.compile`.

## A layout is a shape and a stride

(5) $$\text{offset}(c_0, c_1, \dots) = \sum_k c_k \, s_k \qquad (4,8) : (8,1) : \; (i,j) \mapsto 8i + j$$

- **Layout**: a shape plus a stride
- Written `shape:stride`
- Offset: each coordinate times its stride, summed

| `(4, 8):(8, 1)` | `i = 0` | `i = 1` | `i = 2` | `i = 3` |
|---|---|---|---|---|
| j = 0 | 0 | 8 | 16 | 24 |
| j = 1 | 1 | 9 | 17 | 25 |
| j = 2 | 2 | 10 | 18 | 26 |
| j = 3 | 3 | 11 | 19 | 27 |
| j = 4 | 4 | 12 | 20 | 28 |
| j = 5 | 5 | 13 | 21 | 29 |
| j = 6 | 6 | 14 | 22 | 30 |
| j = 7 | 7 | 15 | 23 | 31 |

▼ From a coordinate to an offset

A CuTe layout maps a logical coordinate to an offset in memory. The shape gives the size of each mode, and the stride gives how far one step along that mode moves. CuTe writes it as `shape:stride`, and the offset is the sum of each coordinate times its stride (Formula 5).

A row-major tile of 4 rows and 8 columns has the layout `(4, 8):(8, 1)`: a step along a row (`j`) moves one element, a step down a column (`i`) moves eight, so `(i, j)` maps to `8i + j`. Coordinate `(2, 5)` maps to `2 · 8 + 5 = 21`. The table lists all 32 offsets, with the tile's 4 rows as its columns. With stride `(1, 4)` the same shape is column-major, `(i, j) ↦ i + 4j`: changing the stride changes the storage order. Code that has a layout does not compute addresses by hand. It applies the layout, and CuTe composes layouts into new ones.

## Thread-value layouts are layouts too

- **Thread-value layout**: (thread t, value v) → offset
- Course kernel: `(1024, 4):(4, 1)`
- PyTorch's kernel: `(1024, 4):(1, 1024)` (PyTorch Contributors, 2026)
- Both cover each offset once, both coalesced

| | `(1024, 4):(4, 1)` | `(1024, 4):(1, 1024)` |
|---|---|---|
| Offset | `4t + v` | `t + 1024v` |
| Thread 0 | 0, 1, 2, 3 | 0, 1024, 2048, 3072 |
| Thread 1 | 4, 5, 6, 7 | 1, 1025, 2049, 3073 |
| Thread 31 | 124, 125, 126, 127 | 31, 1055, 2079, 3103 |
| Thread 1023 | 4092, 4093, 4094, 4095 | 1023, 2047, 3071, 4095 |
| Loads per thread | one of 8 bytes | four of 2 bytes |
| One warp, one load | 256 contiguous bytes | 64 contiguous bytes |

▼ **Two layouts for the same row, and what one warp reads**

A thread-value layout says which thread handles which element. Its first mode is the thread `t`, its second the value `v` among that thread's elements. For a 4,096-wide row and 1,024 threads the shape is `(1024, 4)`, and the stride decides where the values are.

Stride `(4, 1)` gives `4t + v`: thread `t` owns the adjacent offsets `4t` to `4t + 3`. This is the course kernel's default, `1024x4`. Stride `(1, 1024)` gives `t + 1024v`: thread `t` owns `t`, `t + 1024`, `t + 2048` and `t + 3072`. The offsets in the table were printed from the compiled layouts on UC3's H100 and match for every thread.

The second layout is PyTorch's. For a 4,096-wide bf16 row, PyTorch's CUDA softmax launches one thread block (CTA) of 1,024 threads per row, keeps the row in registers, and loads one element at a time at `offset = threadIdx.x + reg_idx * blockDim.x` (PyTorch Contributors, 2026), which with `blockDim.x = 1024` is `t + 1024v`. The course script includes it as the `strided` variant.

Both layouts cover each offset once and are coalesced, in different ways. Under `4t + v` a thread's four values are 8 adjacent bytes, and the compiled code loads them with one 8-byte instruction ( `LDG.E.64` ), so a warp reads 256 contiguous bytes, eight sectors, per instruction. Under `t + 1024v` the compiled code issues four 2-byte loads ( `LDG.E.U16` ), and at each one the warp reads 64 contiguous bytes, two sectors. They differ in the bytes per load instruction, the second lever of slide 13. A third layout, `(512, 8):(8, 1)`, the script's `512x8`, gives each

of 512 threads eight adjacent values, 16 bytes, loaded with one `LDG.E.128`, the widest load an H100 thread can issue.

18

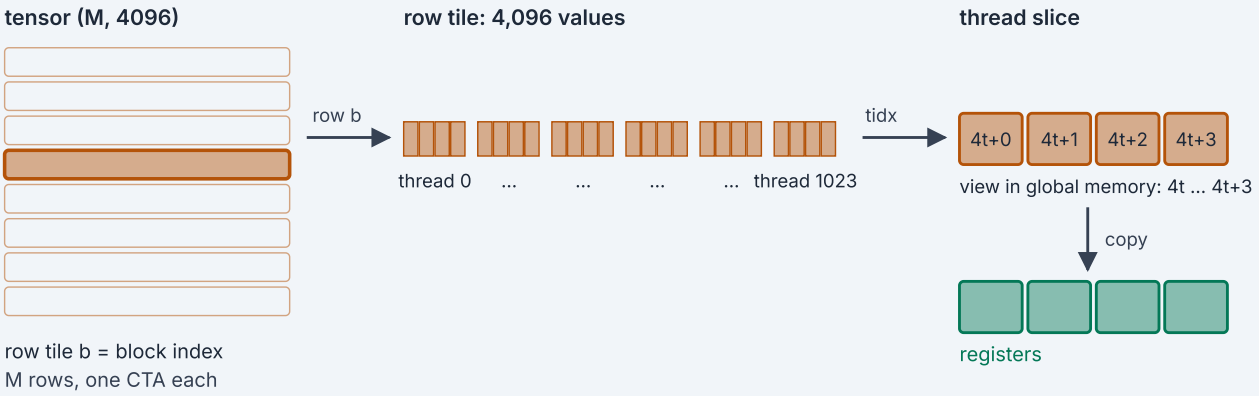
## Which offsets does thread 5 own

### under (1024, 4):(4, 1) and (1024, 4):(1, 1024)?

Answer A2

# Tile by rows, then slice by thread

- `zipped_divide` by `(1, 4096)`: M row tiles
- `bidx`: this CTA's row
- `composition` with the layout, then `[tidx, None]`




`zipped_divide` cuts the tensor by the tiler `(1, 4096)` into M row tiles, and the block index picks this CTA's row. Composing that row with the thread-value layout and indexing it with `[tidx, None]` gives the four values this thread owns, grouped by 4 so that thread t owns the group starting at offset 4t. That slice, `tXgX`, is only a view of those four elements in global memory. The copy (`make_rmem_tensor` and `autovec_copy`) is what moves them into registers.

▼ **Two divisions: from the tensor to one thread's values**

`zipped_divide(mX, (1, 4096))` divides the `(M, 4096)` tensor by the tiler `(1, 4096)`, one full row. The result has shape `((1, 4096), (M, 1))`: the first mode indexes elements inside a tile, the second the `M` tiles. The host launches one CTA per tile, and in the kernel `gX[((None, None), bidx)]` keeps the whole first mode and fixes the tile to the CTA's index `bidx`, a `(1, 4096)` view of this CTA's row. The tiler needs both modes. A one-mode tiler `(4096,)` would divide the row-count mode instead, and each tile would be a column of 4,096 rows.

`cute.composition(row, tv_layout)` re-indexes the row through the thread-value layout, so element `(t, v)` of the result is row element `4t + v`, and `[tidx, None]` fixes this thread and keeps all its values. In order: `tv_layout` defines ownership, `composition` applies it to the row, and indexing gives this thread's slice. The kernel never computes an address by hand.

# Fused Softmax

20

## Each row needs a maximum, a sum and a scale

(6) $$\mathrm{softmax}(x)_j = \frac{e^{x_j - m}}{l}, \qquad m = \max_k x_k, \qquad l = \sum_{k=1}^{N} e^{x_k - m}$$

| Step | Eager chain | Fused kernel | Walkthrough |
|---|---|---|---|
| Launch, load the row | 5 kernels | 1 CTA per row, 1 read into registers | 1, 2 |
| Maximum $m$ | 1 read | reduction in three tiers | 3 |
| Exponentials $e^{x_j - m}$ | 2 reads, 2 writes | `exp2` on the SFU, in registers | 3 |
| Sum $l$ | 1 read | reduction in three tiers, in fp32 | 3 |
| Divide, store | 1 read, 1 write | 1 reciprocal, 1 bf16 write | 4 |

- Subtracting $m$ keeps every exponential in $[0, 1]$
- Walkthroughs 1 to 4: the kernel's code, in order

▼ What a softmax kernel has to do

### The definition

The softmax turns a row $x = (x_1, \ldots, x_N)$ of logits, here $N = 4{,}096$ bf16 values, into probabilities. Formula (6) writes it in the form kernels compute. Subtracting the row maximum $m$ changes nothing, because it multiplies the numerator and the denominator by the same factor $e^{-m}$:

$$\frac{e^{x_j - m}}{\sum_k e^{x_k - m}} = \frac{e^{-m}\, e^{x_j}}{e^{-m} \sum_k e^{x_k}} = \frac{e^{x_j}}{\sum_k e^{x_k}}.$$

This is an exact identity. After the subtraction every exponent is at most 0, so every exponential lies in $[0, 1]$ and none can overflow.

### Three steps, two of them reductions

Formula (6) fixes the order of the work. The maximum $m$ needs the whole row. The sum $l$ needs $m$ and, again, the whole row. Only then can each value be divided by $l$. So a kernel makes two reductions over the row, one after the other, and then scales every value. The

course kernel follows this order with two sweeps over the row. The online softmax of slide 28 merges the two reductions into one.

The table compares the eager chain of slide 4 with the fused kernel, step by step. Run as five PyTorch operations, the steps read the row five times and write three times, two intermediates and the output, eight passes through HBM. The fused kernel reads the row once into registers and writes the result once. In between, it has to make each step cheap on the chip:

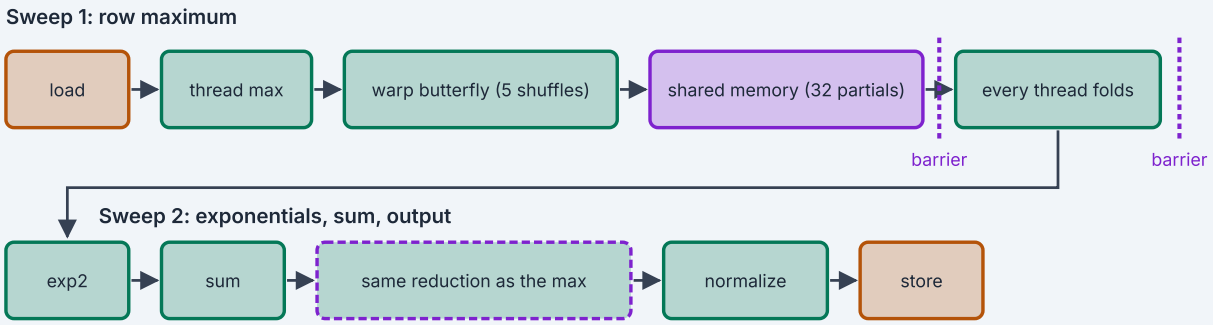
- The reductions. The 4,096 values of a row are spread over the 1,024 threads of one CTA, four per thread, so no thread sees the whole row. The threads combine their values in three tiers: each thread over its own values, each warp through register shuffles (slide 11), and the warps through shared memory (slide 12).
- The exponentials. The special-function unit computes powers of 2, so the kernel computes $e^{x_j - m}$ as a power of 2 (Formula 7).
- The sum. It is accumulated in fp32, because a long sum in bf16 loses its small terms (slide 26).
- The division. Each thread computes one reciprocal of $l$ and multiplies its four values by it, instead of dividing four times.
- The load and the store. In the default layout, each thread moves its four values with one vectorized, coalesced access (slide 14).

### How the walkthroughs read the code

The kernel lives in the course script `scripts/kernels/softmax_cute.py`. Slide 21 draws its data flow, and Walkthroughs 1 to 4 then read its code in the order it runs, as the last column of the table shows. The note on Walkthrough 1 says which function each block comes from.

# One CTA per row, the row kept in registers

- Global memory: one read, one write
- Registers: the row, through both sweeps
- Shared memory: 32 warp partials only




Global memory (amber) is touched twice: one load of the row and one store of the output. Registers (green) hold each thread's four values, the row max and the row sum, and every butterfly step. The row stays in registers between the two sweeps. Shared memory (violet) holds only the 32 per-warp partials. Lane 0 of each warp writes one partial, and the barrier makes all 32 visible before any thread reads them.

### ▼ The kernel as one picture

A CTA of 1,024 threads handles one row. It reads the row once from global memory, four bf16 values per thread, and keeps it in fp32 registers. The kernel then makes two sweeps over the row in registers, not in HBM: the first finds the row maximum, the second computes the exponentials and their sum. Each sweep reduces in three tiers, and shared memory holds only the 32 warp partials, 128 bytes, between two barriers. The output is normalized and written once.

PyTorch's CUDA kernel for a 4,096-wide bf16 row has the same shape: one thread block of 1,024 threads per row, the row in registers, warp partials combined in shared memory, and one read of the input (PyTorch Contributors, 2026). QuACK's softmax chooses differently. Its 128-thread CTAs handle two rows each, 64 threads per row. Each thread loads 16 bytes at a time, the data goes into shared memory through asynchronous copies ( `cp.async` ) and then into registers, and the maximum and the sum are computed together, the online softmax of slide 28 (Dao-AILab, 2026). Which mapping works best depends on the row width and the row count.

## Walkthrough 1: the host launches M CTAs

- Host function `softmax_host`: runs once per call
- Builds the layout, tiles the rows, launches the kernel

Global memory (HBM) Registers Shared memory

```python
@cute.jit
def softmax_host(mX: cute.Tensor, mO: cute.Tensor, stream: cuda.CUstream):
    # Thread-value layout: (thread t, value v) -> offset within the row.
    tv_layout = cute.make_layout((threads, vec), stride=tv_stride)
    # Cut the (M, 4096) tensors into M row tiles of shape (1, 4096).
    gX = cute.zipped_divide(mX, (1, SUPPORTED_N))
    gO = cute.zipped_divide(mO, (1, SUPPORTED_N))
    # One CTA per row tile.
    softmax_kernel(gX, gO, tv_layout).launch(
        grid=[cute.size(gX, mode=[1]), 1, 1],
        block=[threads, 1, 1],
        stream=stream,
    )
```

▼ **The host function, line by line**

### Where each block comes from

The script defines three DSL functions, and the walkthroughs show each one from its decorator and typed signature onward. `softmax_host`, on this slide, is decorated with `@cute.jit` and runs on the host. `softmax_kernel`, decorated with `@cute.kernel`, is the device code that every thread runs. Walkthroughs 2, 3 and 4 show it in three consecutive parts, from its header to its last line. `row_reduce`, the second block of Walkthrough 3, is a `@cute.jit` helper that the kernel calls twice. All three are nested inside the function that builds one variant of the kernel, which sets the constants `threads`, `vec`, `tv_stride`, `num_warps` and `log2_e`. The blocks keep the script's indentation, so they match the script line for line.

### The host function

`mX` and `mO` are the whole `(M, 4096)` input and output tensors in global memory, and `stream` is the CUDA stream to launch on. The first line of the body builds the thread-value layout from the variant's constants. For the default, `threads = 1024`, `vec = 4` and `tv_stride = (4, 1)`, giving `(1024, 4):(4, 1)` of slide 17. The next two lines tile the input `mX` and the output `mO` into row tiles, as on slide 19. Both stay in global memory, as the amber marks show. The prefix `m` names a whole tensor and `g` a tiled view of global memory.

The launch passes the tiled tensors and the layout to the kernel. The grid has `cute.size(gX, mode=[1])` CTAs, the size of the tile mode, which is `M`, and each CTA has `threads` threads.

The stream argument puts the launch on PyTorch's current CUDA stream, in order with the surrounding operations.

## Walkthrough 2: each thread loads four values

- Kernel `softmax_kernel` begins: every thread runs it
- Each thread loads its four values once

Global memory (HBM) Registers Shared memory

```python
@cute.kernel
def softmax_kernel(gX: cute.Tensor, gO: cute.Tensor, tv_layout: cute.Layout):
    # This thread's index in its CTA, and the CTA's index in the grid.
    tidx, _, _ = cute.arch.thread_idx()
    bidx, _, _ = cute.arch.block_idx()
    # One fp32 slot per warp in shared memory, for the warp partials.
    smem = cutlass.memory.SmemAllocator()
    reduction_buffer = smem.allocate_tensor(
        Float32, cute.make_layout(num_warps), byte_alignment=4
    )
    # This CTA's row (block index bidx), then this thread's values in it.
    tXgX = cute.composition(gX[((None, None), bidx)], tv_layout)[tidx, None]
    tXgO = cute.composition(gO[((None, None), bidx)], tv_layout)[tidx, None]
    # Copy the bf16 values from global memory into registers, widen to fp32.
    frgX = cute.make_rmem_tensor(cute.size(tXgX), gX.element_type)
    cute.autovec_copy(tXgX, frgX)
    x = frgX.load().to(Float32)
```

▼ **Each call in the load, and CuTe's names**

The block starts at the kernel's header. Its parameters are the two tiled tensors and the thread-value layout that the host passed to the launch. The first lines read `tidx`, this thread's index in its CTA, and `bidx`, the CTA's index in the grid, and allocate `reduction_buffer` in shared memory: one fp32 slot per warp, 32 for the default variant, which both reductions use. `num_warps`, used here and in `row_reduce`, and `log2_e`, used in Walkthrough 3, are Python constants of the enclosing build function.

The two `composition` lines are slide 19 in code. `tXgX` is a view of this thread's four input elements in global memory and moves no data, and `tXgO` is the matching view of the output. `cute.make_rmem_tensor` allocates a four-element bf16 tensor in registers. `cute.autovec_copy` copies the global view into it and vectorizes the copy where the layout allows. In the default layout the four values are 8 adjacent bytes, and the compiled code moves them with one 8-byte load. `.load()` turns the register tensor into a `TensorSSA`, the DSL's value type for arithmetic on whole fragments, and `.to(Float32)` widens the values, so `x` holds them in fp32 registers.

CuTe's names encode where data lives. In `tXgX`, `t` marks a thread's partition, `X` the tensor partitioned, and `gX` the tensor in global memory: this thread's slice of `X` in global memory.

`frg` marks a register fragment, so `frgX` is the input fragment in registers. In the same scheme `s` marks shared memory and `r` registers.

## Walkthrough 3: two reductions in three tiers

- Kernel continued: maximum, exponentials, sum
- Helper `row_reduce`, called twice: three-tier reduction

Global memory (HBM) Registers Shared memory

```
    row_max = row_reduce(x, cute.ReductionOp.MAX, reduction_buffer, -Float32.inf)
    # e^(x - max) computed as 2^((x - max) * log2 e) on the special-function unit.
    exp_x = cute.math.exp2(x * log2_e - row_max * log2_e, fastmath=True)
    row_sum = row_reduce(exp_x, cute.ReductionOp.ADD, reduction_buffer, Float32(0.0))


@cute.jit
def row_reduce(x: cute.TensorSSA, op: cutlass.Constexpr, reduction_buffer: cute.Tensor,
               init_val: cute.Numeric) -> Float32:
    fold = {cute.ReductionOp.MAX: cute.arch.fmax, cute.ReductionOp.ADD: operator.add}[op
    # Tier 1: combine this thread's own values.
    val = x.reduce(op, init_val=init_val, reduction_profile=0)
    # Tier 2: a butterfly over the warp's 32 lanes, five register shuffles.
    for i in cutlass.range_constexpr(5):
        val = fold(val, cute.arch.shuffle_sync_bfly(val, offset=1 << i))
    # Tier 3: lane 0 of each warp writes its partial to shared memory.
    # Read both indices before the branch: warp_idx() is a warp-wide
    # shuffle, which deadlocks if only lane 0 executes it.
    lane = cute.arch.lane_idx()
    warp = cute.arch.warp_idx()
    if lane == 0:
        reduction_buffer[warp] = val
    cute.arch.barrier()
    # Every thread folds all partials, so every thread holds the row result.
    acc = reduction_buffer[0]
    for w in cutlass.range_constexpr(1, num_warps):
        acc = fold(acc, reduction_buffer[w])
    # Before anyone writes the buffer again, every thread must finish reading it.
    cute.arch.barrier()
    return acc
```

▼ **Thread, warp and CTA tiers, and why the second barrier is needed**

The first block continues the kernel right after the load of Walkthrough 2. It calls `row_reduce` twice, with `MAX` and with `ADD`, and both calls use the same 32-slot shared-memory buffer.

In `row_reduce` (second block, the whole helper), the first line of the body picks the combining function, `fmax` or addition. The choice is made at compile time, because `op` is a compile-time constant. Tier 1: `x.reduce` combines the thread's four values into `val`. Tier 2: in step `i` of the butterfly, each lane combines its `val` with that of the lane whose index differs in bit `i`, for

offsets 1, 2, 4, 8 and 16. After the first step each pair of lanes agrees, after the second each group of four, and after five steps all 32. The compiled code shows the five shuffles ( `shfl.sync.bfly` ) in each reduction. The H100 has no single instruction for an fp32 warp reduction: the fp32 form of the warp-reduction instruction exists only on sm_100-family (data-center Blackwell) GPUs, and only for minimum and maximum (NVIDIA, 2026c). QuACK uses the same five shuffles for fp32 on the H100 (Dao-AILab, 2026).

Tier 3 crosses warps through shared memory. Lane 0 of each warp writes its result to `reduction_buffer[warp]` . Both indices are read before the `if` , because `cute.arch.warp_idx()` reads the warp index with a warp-wide shuffle (the compiled code shows `shfl.sync.idx` ), which every lane must reach (slide 11). The first barrier waits until all 32 partials are written. Then every thread folds all 32 into `acc` , so every thread holds the row result and none has to broadcast it.

The second barrier protects the buffer. Without it, a fast warp could finish the maximum, compute its exponentials and write a partial sum into the buffer while a slow warp is still reading the maximum's partials, which would then fold a sum into its maximum. The compiled code has both barriers ( `bar.sync` ) in each reduction.

## Walkthrough 4: normalize in fp32, store bf16 once

- Kernel ends: scale, narrow to bf16, store once

(7) $$e^{x-m} = 2^{(x-m)\log_2 e}$$

Global memory (HBM) Registers Shared memory

```
# Normalize in fp32, narrow to bf16, write the row back once.
y = exp_x * cute.arch.rcp_approx(row_sum)
frg0 = cute.make_rmem_tensor(cute.size(tXg0), g0.element_type)
frg0.store(y.to(g0.element_type))
cute.autovec_copy(frg0, tXg0)
```

▼ **The exponential, the reciprocal and the store**

The exponential is computed between the two reductions of slide 24: `exp_x = cute.math.exp2(x * log2_e - row_max * log2_e, fastmath=True)`. The special-function unit computes powers of 2, so the kernel rewrites `e^(x - m)` as `2^((x - m) · log₂ e)` (Formula 7). Since `m · log₂ e` is the same for the whole row, this costs one multiply-add per value. `fastmath=True` selects the fast approximate instruction. Subtracting the maximum keeps every exponential in `[0, 1]`, where masked `-inf` entries give 0.

This block is the end of the kernel. `cute.arch.rcp_approx(row_sum)` computes one approximate reciprocal per thread, so each thread does one reciprocal and four multiplications instead of four divisions. The last three lines mirror the load. `frg0` is a bf16 register fragment, and `frg0.store(y.to(...))` narrows the fp32 result `y` into it. `autovec_copy` then writes it through `tXg0` into the output row, and the compiled code does this with one 8-byte store per thread. All arithmetic in between stayed in fp32 registers.

Every variant passed the correctness check against PyTorch's softmax computed in fp32 and rounded to bf16, at bf16 tolerances. The check covers five patterns of values, including rows with `-inf` entries, and four row counts.

## fp32 accumulation protects the normalizer

$$\underbrace{1+1+\cdots+1}_{4{,}096\ \text{terms}} = \begin{cases} 256 & \text{summed in order in bf16} \\ 4{,}096 & \text{in fp32} \end{cases} \tag{8}$$

- bf16, fp32: same 8-bit exponent
- After max subtraction: exponentials in [0, 1]
- Risk: rounding in long sums
- Constructed example: bf16 running sum stalls at 256

▼ What bf16 shares with fp32, and where it falls short

The kernel loads bf16 but computes the exponentials and both reductions in fp32. bf16 and fp32 both have 8 exponent bits, so they cover about the same range of magnitudes, and after the maximum is subtracted every exponential lies in `[0, 1]`. They differ in precision. Recall from Week 5 that bf16 keeps 7 mantissa bits, so in `[2^k, 2^(k+1))` neighboring bf16 numbers are `2^(k-7)` apart.

Formula (8) is a constructed example, checked with PyTorch on a CPU. A row of 4,096 equal logits has every exponential equal to 1, so the sum should be 4,096. Add the ones one by one in a bf16 accumulator. Up to 256 every sum is exact. In `[256, 512)` the spacing is 2, so `256 + 1 = 257` lies halfway between 256 and 258, and round-to-nearest-even picks 256. Every further addition rounds back to 256. Each probability then comes out as `1/256` instead of `1/4096`, 16 times too large, and the row sums to 16. An fp32 accumulator gives 4,096, and so does `torch.sum` on a bf16 tensor, because PyTorch accumulates bf16 sums in fp32.

The kernel's tiers add in a tree, which loses less than this running sum, but the principle holds: accumulate the normalizer in fp32 and narrow to bf16 once, at the store. The correctness check includes rows of equal values, and every variant passes it.

## On UC3's H100, 512x8 and QuACK reach copy speed

| Implementation | Median µs | GB/s | % of copy | % of peak | Spread | Kernels |
|---|---|---|---|---|---|---|
| Copy ( `out.copy_(x)` ) | 251.7 | 2,132.9 | 100.0% | 89.0% | 0.0% | 0 (1 copy) |
| Eager chain | 1,248.0 | 430.2 | 20.2% | 18.0% | 0.5% | 5 |
| `torch.softmax` | 466.8 | 1,150.2 | 53.9% | 48.0% | 24.3% | 1 |
| `torch.compile` | 359.3 | 1,494.1 | 70.1% | 62.4% | 6.9% | 1 |
| Course `1024x4` | 369.4 | 1,453.5 | 68.1% | 60.7% | 2.1% | 1 |
| Course `strided` | 377.6 | 1,421.8 | 66.7% | 59.3% | 5.6% | 1 |
| Course `512x8` | 253.5 | 2,118.0 | 99.3% | 88.4% | 0.2% | 1 |
| QuACK | 254.5 | 2,109.5 | 98.9% | 88.0% | 0.1% | 1 |

- Compiler: five kernels into one
- Layout: 2% gap, within the 5.6% spread
- 512×8: 16-byte loads, half the warps

▼ How we measured, and what each comparison shows

The measurement ran on UC3's H100, which `nvidia-smi` reports as "NVIDIA H100" with 95,830 MiB, with driver 615.71.09, PyTorch 2.13.0 built for CUDA 13.0, `nvidia-cutlass-dsl` 4.8.0 and QuACK 0.6.5, on 2026-10-10. The input is a 32,768 × 4,096 bf16 tensor, 268 MB, and every row of the table takes bf16 input and returns bf16 output.

Correctness came first: all three course variants passed the check of slide 25 for 1, 8, 1,000 and 1,024 rows. The benchmark rotates through four 268 MB input buffers, 1,074 MB in total, far more than the card's 60 MiB L2. CUDA events time a loop of 100 back-to-back calls after 10 warm-up calls, the loop runs five times, and the table gives the median time per call. The spread is the range of the five loop times, as a percentage of the median. Effective bandwidth divides the same ideal byte count, one read and one write (2 × 268 MB), by the median for every row (NVIDIA, 2024a), so a row that moves extra bytes shows a lower number. "% of copy" compares with the copy row, the practical ceiling on this card, and "% of peak" with the theoretical 2,395.9 GB/s of slide 8. The PyTorch profiler counted the kernels of one call. The copy is a memory copy and launches no kernel. As a cross-check, the summed device time of a single profiled call agrees with the medians within 12% (242.9 µs for the copy, 243.5 µs for `512x8` ).

The GPU's clock was not constant. During most rows the driver reported a software power cap (throttle reason 0x4), and the SM clock read right after each row ranged from 1,545 MHz, after

`torch.softmax`, to the maximum of 1,980 MHz, after the copy and QuACK. Each reading is one sample. They show that rows ran at different SM clocks, not how much each row was slowed.

`torch.compile` turned five kernels into one, at 359.3 µs, 70.1% of the copy and 1,248.0 / 359.3 ≈ 3.5 times faster than the eager chain. Without any kernel code, it was slightly faster than the course's walkthrough kernel at 369.4 µs. This is why slide 6 tries the compiler first.

The layout comparison is not resolved by this run. `1024x4` and `strided` differ only in the layout of slide 17: one 8-byte load per thread against four 2-byte loads, both coalesced. Their medians, 369.4 and 377.6 µs, differ by about 2%, less than the strided row's 5.6% spread. `torch.softmax`, which uses the strided mapping, had a 24.3% spread and the lowest clock reading, so its median of 466.8 µs supports no argument about layout.

The two kernels with 16-byte loads reach the copy's speed. `512x8` takes 253.5 µs, 99.3% of the copy, against 369.4 µs and 68.1% for `1024x4`. The compiled code of all three course variants uses 32 registers per thread, so four `512x8` CTAs of 512 threads fit on an SM, against two `1024x4` CTAs of 1,024: both hold 2,048 threads, 64 warps, the same occupancy. Each `512x8` thread loads 16 bytes in one instruction, so when its CTAs load at once the SM has 2,048 × 16 bytes = 32 KB in flight, against 16 KB for `1024x4`, the second lever of slide 13. The variant also changes a second thing: each CTA has 16 warps instead of 32, so each cross-warp fold combines 16 partials. This run does not separate the two effects. QuACK reaches the same speed, 254.5 µs and 98.9%, with the different design of slide 21: two rows per 128-thread CTA, 16-byte asynchronous copies through shared memory and the online softmax.

The table measures one shape, one dtype and one GPU. Another setting needs its own measurement.

# To FlashAttention

28

## Attention streams its row and carries m, l, o

(9) per K/V block

$$m_{\text{new}} = \max\left(m_{\text{old}},\ \max_j s_j\right)$$
$$l_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}}\, l_{\text{old}} + \sum_j e^{s_j - m_{\text{new}}}$$
$$o_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}}\, o_{\text{old}} + \sum_j e^{s_j - m_{\text{new}}}\, v_j$$

(10)
$$\text{output} = o/l \quad \text{(once, after the last block)}$$

- Row too long to keep: stream K/V blocks
- Carried state: running max, sum, output
- Rescale `l` and `o` when `m` grows

▼ From two sweeps over a resident row to one streaming pass

The course kernel keeps its whole row in registers, so it can sweep over it twice, for the maximum and then for the exponentials and their sum, while reading global memory once. An attention row has one score per key, and at realistic sequence lengths the row's scores and the key and value blocks they need do not all fit on the chip. FlashAttention therefore streams the K/V blocks past each query row in a single pass and carries a small state from block to block (Dao et al., 2022).

Write `s_j` for the score of key `j` and `v_j` for its value row. The state is Week 4's: `m`, the largest score so far; `l`, the running sum of exponentials, the softmax normalizer; and `o`, the accumulator `Σ_j exp(s_j - m) v_j`, not yet divided by `l`. The state starts at `m = -∞`, `l = 0` and `o = 0`. Every term in `l` and `o` was computed against `m_old`, and multiplying by `exp(m_old - m_new)` turns `exp(s_j - m_old)` into `exp(s_j - m_new)`. So each new block first rescales the old `l` and `o` by that factor and then adds its own terms (Formula 9). If the block does not raise the maximum, the factor is 1. After the last block, `o / l` equals the attention output for the row in exact arithmetic (Formula 10).

FlashAttention-1 divides the output by the normalizer at every step (Dao et al., 2022). FlashAttention-2 keeps the accumulator unscaled and divides once, at the end, saving work in the loop (Dao, 2024), and Formula (9) follows that form. Either way the score and probability matrices never reach HBM, and the result is exact attention. The online form also helps a single

row that fits on the chip: QuACK's softmax carries the maximum and the sum together, so one exchange through shared memory replaces the course kernel's two (Dao-AILab, 2026).

29

## What are m, l, o after each tile?

**Tile 1: scores (1, 2), values (1, 0)**
**Tile 2: scores (3, 0), values (2, 1)**

Answer A3

30

## FlashAttention adds Tensor Cores and pipelining

- Q, K, V tiles sized for shared memory
- Tensor Cores: matrix multiply-accumulate (MMA) instructions
- Hopper: four warps, one warpgroup, one MMA
- Tensor Memory Accelerator (TMA): asynchronous tile copies
- Warp specialization: producer and consumer warps

▼ **Tiles, matrix units and a pipeline**

FlashAttention splits Q, K and V into tiles that fit in shared memory and computes $QK^T$ and the product of the probabilities with $V$ on Tensor Cores, the matrix units of Week 4, one in each sub-partition of slide 7. A kernel uses them through matrix multiply-accumulate (MMA) instructions, which a whole warp or a group of warps issues together. On Hopper the largest MMA instructions are issued by a warpgroup, four contiguous warps whose first warp's rank is a multiple of 4 (NVIDIA, 2026c) (Andersch et al., 2022). The softmax kernel has no matrix product and uses no Tensor Cores.

Data movement overlaps with this arithmetic. Hopper's Tensor Memory Accelerator (TMA), the unit below the sub-partitions on slide 7, copies large blocks between global and shared memory asynchronously (Andersch et al., 2022), so the next K/V tile arrives while the current one is in use. With warp specialization, producer warps only issue copies, consumer warps only compute, and shared-memory buffers pass the tiles between them. The schedule and the data movement change, and the result is still exact attention.

## FlashAttention-4 rebalances the work for Blackwell

| Per SM, per clock | Hopper | Blackwell B200 |
|---|---|---|
| bf16 MMA operations | 4,096 | 8,192 |
| Exponentials | 16 | 16 |
| Shared memory bytes | 128 | 128 |

- Matrix units doubled, special-function unit unchanged (Zadouri et al., 2026)
- 10–25% of exponentials on fused multiply-add units
- Rescale only for max increases above 8

▼ Two softmax changes for a GPU whose matrix units outgrew the rest

From Hopper to Blackwell, Tensor Core throughput doubled while other units grew more slowly or not at all (Zadouri et al., 2026). Per SM and clock, the B200 performs 8,192 bf16 MMA operations against Hopper's 4,096. Its special-function unit still computes 16 exponentials, and its shared memory still delivers 128 bytes, as on Hopper. The matrix products of attention got faster and the exponentials did not, so the exponentials take a larger share of the time.

FlashAttention-4 changes the softmax in two ways. First, it computes `2^x` in software on the fused multiply-add (FMA) units for 10 to 25% of the entries in each row. A range reduction and a polynomial of degree 3 match the hardware result to within one bf16 unit in the last place. The FMA units and the special-function unit then work in parallel.

Second, it rescales only when the maximum grows by more than a threshold `τ`, typically $\log_2 256 = 8$. The threshold is in base-2 units, because FlashAttention-4, like our kernel, evaluates the exponential as `2^x`. A stale maximum lets exponentials reach at most 256, which fp32 holds easily. Because `l` and `o` use the same stale maximum, it cancels in `o / l`. To avoid divergence, all lanes of a warp rescale together when any one needs it (Zadouri et al., 2026).

32

## Read production kernels by their roles

- Four roles: layout, partition, reduction or MMA, pipeline
- QuACK: `_threads_per_row`, `row_reduce`, clusters (Dao-AILab, 2026)
- FlashAttention-4: tiles, MMA, rescaling, producer warps (Zadouri et al., 2026)

▼ A guide for optional source reading

To go further, label each block of a production kernel with one of four roles: layout (which thread owns which data), partition (how the tensor is cut into tiles and slices), reduction or MMA (the arithmetic) and pipeline (how data moves between tiers and overlaps with compute). Recognizing the roles is enough.

In QuACK's softmax, `_threads_per_row` picks 64 threads for a 4,096-wide row, and CTAs have 128 threads for rows up to 16K wide, so one CTA handles two rows. The vector width is 128 bits divided by the element width, eight bf16 values. `row_reduce` in `reduce.py` combines the thread, warp and CTA tiers of slide 24. Very wide rows are split across a cluster of CTAs, which for 16-bit rows starts above 16K elements, so at 4,096 the cluster has one CTA (Dao-AILab, 2026). In FlashAttention-4, find the tile layouts, the MMA instructions, the softmax with its conditional rescaling, and the producer and consumer warps (Zadouri et al., 2026).

33

## So why are five PyTorch ops 4.9× slower than one kernel?

Answer A4

## Reflection questions

No submission. Bring short answers to the Week 6 tutorial.

1. An eager RMSNorm in bf16 squares `x`, takes each row's mean, applies `rsqrt`, multiplies `x` by the result and then by the weight `g`, one kernel per step. Count its full-tensor passes against a fused kernel and state the ceiling with its assumptions. The course's `RMSNorm` upcasts `x` to fp32 before these steps and casts the result back. How does that change the count and the ceiling?
2. A row-wise kernel such as softmax meets rows of 128 bf16 values in one model and rows of 262,144 in another. Would you give each row a warp, a CTA or a cluster of CTAs? Argue from useful lanes, bytes in flight, registers and shared memory.
3. In bf16, a running sum of 4,096 ones stops at 256. Name two other places in one training step of the course model where a long sum in bf16 would lose precision in the same way, and say how the course code or PyTorch keeps each sum accurate.

# References

35

## References

### Attention kernels


- Dao, T. (2024). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. International Conference on Learning Representations (ICLR 2024).
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.
- Zadouri, T., Hoehnerbach, M., Shah, J., Thakkar, V., & Dao, T. (2026). FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. *MLSys 2026*.


### Kernel sources and engineering notes


- Dao-AILab (2026). QuACK: A Quirky Assortment of CuTe Kernels, version 0.6.5. GitHub repository.
- Guo, W., Zadouri, T., & Dao, T. (2025). Getting Memory-bound Kernels to Speed-of-Light. Dao-AILab engineering note.
- PyTorch Contributors (2026). aten/src/ATen/native/cuda/SoftMax.cu. PyTorch source code, GitHub.


### GPU architecture and tuning


- Andersch, M., Palmer, G., Krashinsky, R., Stam, N., Mehta, V., Brito, G., & Ramaswamy, S. (2022). NVIDIA Hopper Architecture In-Depth. NVIDIA Technical Blog.
- NVIDIA (2022). NVIDIA H100 Tensor Core GPU. Product specifications.
- NVIDIA (2023). NVIDIA Hopper Tuning Guide. CUDA Toolkit documentation, accessed 2026-07-20.
- NVIDIA (2024a). CUDA C++ Best Practices Guide. CUDA Toolkit documentation, accessed 2026-07-20.
- NVIDIA (2026a). CUDA Programming Guide. CUDA Toolkit documentation, accessed 2026-10-10.
- NVIDIA (2026b). Nsight Compute Profiling Guide. Nsight Compute documentation, accessed 2026-10-10.
- NVIDIA (2026c). Parallel Thread Execution ISA. CUDA Toolkit documentation, accessed 2026-10-10.
- Volkov, V. (2010). Better Performance at Lower Occupancy. GPU Technology Conference (GTC 2010).


### CUTLASS and compiler integration


- NVIDIA (2024b). NVIDIA CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. Online documentation, accessed 2026-07-20.
- Patel, N., Lazos, M., Guessous, D., & Ellison, E. (2026). Generating State-of-the-Art GEMMs with TorchInductor's CuTe DSL Backend. PyTorch Blog.


36

## Image credits


- Lucotte, J.-R. (draughtsman), Bénard, R. (engraver), for Diderot and d'Alembert (1772). A gold-thread spinning wheel with one great wheel driving sixteen bobbins in parallel, in elevation. *Encyclopédie, Recueil de planches, vol. IX: Fileur d'or, 'Rouet à seize bobines'*, Pl. VII. Bibliothèque nationale de France (via Gallica). Public domain.

# Answers

## A1 · Each eager operation is a round trip through HBM

- Eager chain: five kernels, one per operation
- Each kernel reads and writes HBM
- Fused kernel: one read, one write
- Few FLOPs per byte: bytes set the time

### ▼ Five kernels, five round trips

On UC3's H100, a row softmax over a 32,768 × 4,096 bf16 tensor takes 1,248.0 µs when it is written as five PyTorch operations and 253.5 µs in the fastest single kernel we measured, about a fifth of the time (1,248.0 / 253.5 ≈ 4.9). Both compute the same softmax on the same GPU. So why are the five operations 4.9 times slower than one kernel?

The first part of the answer is where the data goes between the operations. Run eagerly, each of the five operations launches its own kernel, and the profiler counts five kernels per call. A kernel cannot hand its result to the next kernel on the chip. It writes its output tensor to HBM, the GPU's off-chip memory, and the next kernel reads it back. One tensor here is 268 MB, far more than the GPU's on-chip caches hold, so every intermediate makes the full round trip. The single kernel reads the input once, keeps each row on the chip while it computes, and writes the output once.

Softmax does only a few arithmetic operations per byte, so the bytes set its time. Two questions remain: how many bytes each version moves, and whether the byte count accounts for the whole 4.9×.

Back to the question

## A2 · Thread 5 owns adjacent or scattered offsets

- `4t + v`: 20, 21, 22, 23
- `t + 1024v`: 5, 1029, 2053, 3077
- First: 8 adjacent bytes, one load
- Second: four 2-byte loads, 2,048 bytes apart

▼ **Thread 5 under both layouts**

Which row offsets does thread 5 own under the course layout `(1024, 4):(4, 1)`, and which under PyTorch's `(1024, 4):(1, 1024)`?

Apply each layout to the coordinates `(5, v)` for the values `v = 0, 1, 2, 3`. The course layout gives `4 · 5 + v`: offsets 20, 21, 22 and 23. PyTorch's layout gives `5 + 1024v`: offsets 5, 1029, 2053 and 3077. Each element is a 2-byte bf16 value, so under the first layout thread 5 owns bytes 40 to 47 of the row. The compiled code loads these 8 adjacent bytes with one 8-byte instruction. Under the second layout its four values lie 1,024 elements, 2,048 bytes, apart, and each takes its own 2-byte load.

Both layouts stay coalesced. Under the first, threads 4 and 6 own the 8 bytes on either side of thread 5's, and the whole warp reads 256 contiguous bytes in one instruction. Under the second, threads 4 and 6 read the elements next to thread 5's in each of the four loads, and the warp reads 64 contiguous bytes per load. Either way thread 5 owns four values. The stride decides which offsets they are and how many load instructions bring them in.

Back to the question

## A3 · Two rescaled tiles match one softmax

| After | `m` | `l` | `o` |
|---|---|---|---|
| Tile 1 | 2 | 1.368 | 0.368 |
| Tile 2 | 3 | 1.553 | 2.185 |

- Rescale factor: `e^(2 - 3)` ≈ 0.368
- `o / l` ≈ 1.407: same as attention over all four

▼ **Two tiles by hand**

One query row meets four keys in two tiles, K/V blocks of two keys each. Tile 1 holds the scores 1 and 2 with the values 1 and 0, tile 2 the scores 3 and 0 with the values 2 and 1. Each value is a single number here, so `o` is a single number too. What are `m`, `l` and `o` after each tile?

Start with `m = -∞`, `l = 0` and `o = 0`. Tile 1 raises the maximum to `m = 2`. The old `l` and `o` are zero, so the rescale changes nothing, and the tile adds its own terms: `l = e^(1-2) + e^(2-2) = e^(-1) + 1 ≈ 1.368` and `o = e^(-1) · 1 + 1 · 0 ≈ 0.368`.

Tile 2 holds the score 3, which raises the maximum to `m = 3`. Both old sums were computed against the maximum 2, so Formula (9) first multiplies them by `e^(2-3) = e^(-1) ≈ 0.368`. Then the tile adds its terms against the new maximum: `l ≈ 0.368 · 1.368 + e^(3-3) + e^(0-3) ≈ 0.503 + 1 + 0.050 ≈ 1.553` and `o ≈ 0.368 · 0.368 + 1 · 2 + e^(-3) · 1 ≈ 0.135 + 2 + 0.050 ≈ 2.185`. Formula (10) divides once at the end: `o / l ≈ 2.185 / 1.553 ≈ 1.407`.

Check it against the softmax over all four scores at once. With the maximum 3, the exponentials of 1, 2, 3 and 0 are `e^(-2)`, `e^(-1)`, 1 and `e^(-3)`, which sum to about 1.553, and weighting the values 1, 0, 2 and 1 by them gives about 2.185. The ratio is the same 1.407. The agreement is exact: the rescale turned tile 1's `e^(1-2)` into `e^(1-2) · e^(2-3) = e^(1-3)`, the term the softmax over all four scores at once computes. The probabilities themselves, about 0.087, 0.237, 0.644 and 0.032, were never formed.

Back to the question

## A4 · A quarter of the bytes, moved 1.23 times faster

$$\frac{8 \text{ passes}}{2 \text{ passes}} \times \frac{2{,}118.0 \text{ GB/s}}{1{,}721 \text{ GB/s}} \approx 4.9$$

- Eager: 8 passes through HBM, fused: 2
- Eager chain averages 81% of copy speed
- Fused: the row stays in registers
- Copy speed needs enough bytes in flight
- `512x8` and QuACK: 16-byte loads reach it

▼ The answer in two factors

So why are five PyTorch operations 4.9 times slower than one kernel on the same GPU? The gap has two factors: how many bytes each version moves, and how fast it moves them.

The first factor is the bytes. Run eagerly, the five operations are five kernels, and each intermediate tensor goes to HBM and back. That is five full reads and three full writes, eight passes of 268 MB, against one read and one write for a fused kernel (slide 4). Softmax performs about 1.25 FLOP per byte, far left of every ridge, so its time follows the bytes it moves (slide 5). A quarter of the bytes accounts for a factor of 4.

The second factor is the bandwidth each version reaches. Suppose the eager chain moves its eight passes, about 2.15 GB, in 1,248.0 µs. It then averages about 1,721 GB/s, 81% of the 2,132.9 GB/s that a plain copy reaches on UC3's card. The fastest fused kernel, `512x8`, moves its two passes at 2,118.0 GB/s, 99.3% of the copy. The bandwidth ratio is 2,118.0 / 1,721 ≈ 1.23, and the two factors multiply: 4 × 1.23 ≈ 4.9, the measured gap.

A fused kernel removes the intermediate passes because its data stays on the chip. In the course kernel one CTA handles one row. It keeps the row in fp32 registers through both sweeps, the maximum and the exponentials with their sum, and uses shared memory only for the 32 warp partials, so the row is read once and written once (slide 21).

Fusion alone does not reach the copy's speed. `torch.softmax` and the walkthrough kernel `1024x4` are single kernels too, yet they reach 53.9% and 68.1% of the copy. HBM delivers its bandwidth only when enough bytes are in flight to cover its latency (slide 13). Each `512x8` thread loads 16 bytes in one instruction, so its SM can have 32 KB in flight against 16 KB for `1024x4`, at the same occupancy, and it reaches 99.3% of the copy. The variant also halves the warps per CTA, and our run does not separate the two effects. QuACK reaches 98.9% with its own 16-byte design (slide 27).

Removing the passes does not require writing a kernel first. `torch.compile` turned the five eager kernels into one, 3.5 times faster than the eager chain, and a custom kernel is worth its upkeep only for a boundary the compiler leaves in place (slide 6).

Back to the question